Document No. SPV2-FS-001 Revision A (Draft)

SKETCHPAD VERSION 2

FUNCTIONAL SPECIFICATION

Organization, Modularization, Data Flow, and Construction Plan

Prepared for	Barry Kingsbury, owner
Prepared by	Claude (Anthropic), in working sessions with the owner
Date	1 November 2026
Baseline reviewed	Branch `claude/batch-1`, repository `onbostonharbor1/sketchpad`
Status	Draft for owner review. Nothing in this document has been built.

APPROVAL		
Owner		Date

Rev	Date	Description
A	1 Nov 2026	First issue, for review.

CONTENTS

1.0 SCOPE 4
1.1 Identification 4
1.2 System purpose 4
1.3 Document overview 4
1.4 Conventions 4
2.0 REFERENCED DOCUMENTS 5
3.0 CURRENT SYSTEM AND PROBLEM STATEMENT 6
3.1 Description of Version 1 6
3.2 What V1 does well, and V2 shall keep 6
3.3 Problems 6
4.0 CONCEPT OF THE NEW SYSTEM 8
4.1 The central idea 8
4.2 Decisions recorded with the owner 8
4.3 Assumptions (agreed in discussion, open to change) 8
4.4 Earlier answers reconsidered 8
5.0 ORGANIZATION AND MODULARIZATION 9
5.1 Principle of division 9
5.2 Layers of the system 9
5.3 Draw separation, sharpened 10
5.4 Module definitions 10
5.5 Folder layout 11
5.6 Screen layout 12
6.0 FUNCTIONAL REQUIREMENTS 13
6.1 General 13
6.2 Kernel: paths 13
6.3 Generators 14
6.4 Settings schema and controls 15
6.5 Documents, the point table and welds 16
6.6 Commands and undo 17
6.7 Session state and tab switching 18
6.8 Catalog and storage 18
6.9 Adding scripts and objects while the application runs 18
6.10 Script host and the ingestion ladder 19
6.11 Rendering and physical output 20
6.12 Help, notes, menus and the screens 20
7.0 DATA FLOW 21
7.1 Context diagram 21
7.2 Level 1: the seven processes 21
7.3 Level 2: process 2.0, Edit document 22
7.4 Level 2: process 3.0, Evaluate geometry 23
8.0 DATA DICTIONARY 24
9.0 CHANGES FROM VERSION 1 25
10.0 CONSTRUCTION PLAN 26
10.1 Method 26
10.2 Before CP-0: feature lists 26
10.3 Checkpoints 26
11.0 VERIFICATION 29
12.0 OPEN DECISIONS AND ITEMS TO LOOK AT 30
12.1 Decisions needed from the owner 30
12.2 From the other evaluations: worth your attention 30
13.0 GLOSSARY 31
APPENDIX A. TRACEABILITY 32

LIST OF FIGURES

Figure 1. Version 1 components and their Version 2 replacements 9
Figure 2. Structure chart: the bands of V2 and the direction of calls 10

Figure 3. The three layers: definition, geometry, pixels 10
Figure 4. Screen regions, shown on the Figures tab 12
Figure 5. Paths: the array of points underneath every drawing 13
Figure 6. Makers: three rules, one kind of result 13
Figure 7. Shapers: path in, path out 14
Figure 8. Connectors: four rules that produce four familiar objects 14
Figure 9. A shared point: two layers name point 11 16
Figure 10. An attachment: point 14 lies on layer L3's path 16
Figure 11. Evaluation order, and a loop refused at weld time 16
Figure 12. Snapshot history, the current position and the saved position 17
Figure 13. State transitions of a tab and of an open document 18
Figure 14. Adding a script: save the file, and it appears 19
Figure 15. The ingestion ladder 20
Figure 16. Context diagram (Level 0) 21
Figure 17. Level 1 data flow diagram 22
Figure 18. Level 2: Edit document 23
Figure 19. Level 2: Evaluate geometry 23
Figure 20. Checkpoints and their order 26

1.0 SCOPE

1.1 Identification

This document specifies Sketchpad Version 2 (hereafter "V2"), a rebuild of the Sketchpad application described in the repository `onbostonharbor1/sketchpad`. Sketchpad Version 1 ("V1") is the application as it stands on branch `claude/batch-1`.

1.2 System purpose

Sketchpad serves two purposes of equal standing, as recorded by the owner on 27 October 2026:

1. **Templates.** To produce a printable point template for driving nails into wood for string art.
2. **Sketching.** To explore designs by working directly with how objects are represented and how they relate to one another. Objects kept because they are interesting to play with (Circular Parabola Multi, Linked Circles) are first-class and shall not be treated as clutter.

Where a design choice in this document trades one purpose against the other, the trade is stated and left to the owner.

1.3 Document overview

Section 3 describes V1 and the problems V2 must solve. Section 4 states the concept. Section 5 defines the new organization and its modules. Section 6 states the functional requirements. Section 7 gives the data flow diagrams. Section 8 is the data dictionary. Section 9 maps each V1 component to its V2 replacement. Section 10 is the construction plan, built in checkpoints the owner can use and evaluate. Section 11 covers verification. Section 12 lists open decisions and items from other evaluations the owner should look at. Section 13 is the glossary. Appendix A traces problems to requirements and checkpoints.

1.4 Conventions

- "**Shall**" states a requirement. "**Should**" states a recommendation. "**May**" states an option.
- Requirements are numbered `FR-s.n` by section. Problems are `P-n`. Decisions are `D-n`. Checkpoints are `CP-n`. Open items are `O-n`.
- Data flow diagrams follow the Yourdon/DeMarco notation: a circle is a process, a pair of parallel lines open at one end is a data store, a shadowed square is an external entity (terminator), and a labelled arrow is a data flow. A terminator or store drawn twice carries an extra mark.
- The data dictionary uses DeMarco notation: `=` is composed of, `+` and, `[ | ]` one of, `{ }` iterations of, `( )` optional, `* *` comment.
- TBD marks a value the owner has not yet decided.

2.0 REFERENCED DOCUMENTS

Ref.	Title	Where
R-1	Sketchpad: notes for Claude (project rules)	`CLAUDE.md`
R-2	Sketchpad phases	`PHASES.md`
R-3	Not done: items from the phase lists, sections 6 and 7	`docs/NOT_DONE.md`
R-4	Sketchpad: Architecture Rethink (staged)	`docs/Architecture-Rethink.pdf`
R-5	Sketchpad: Rethink from Scratch	`docs/Rethink-From-Scratch.pdf`
R-6	Sketchpad Version 2: Architecture (19 pp.)	supplied by the owner
R-7	Sketchpad Version 2 Proposal (17 pp.)	supplied by the owner
R-8	Backlog Triage and Figures Tab Spec, draft 3, with the owner's answers (23 Oct 2026)	supplied by the owner
R-9	Review table, main to batch-1	`docs/TEST_TABLE_main-to-batch-1.md`

3.0 CURRENT SYSTEM AND PROBLEM STATEMENT

3.1 Description of Version 1

V1 is a browser application in plain JavaScript modules, served by Vite on port 5173, with a local Node service on port 5174 that performs every file write. It has six tabs: Home, Draw, Patterns, Figures, Gallery and Utilities. About 27,000 lines are the owner's own code (excluding `gallery/`, `patterns/` and the vendored TinyMCE). The library holds 14 registered Draw objects and about 185 pattern and gallery scripts.

V1 began as a program that runs a drawing script on a canvas. Since Phase 6 it has grown into a geometry editor: objects on layers, discs on infrastructure lines, welds, layer transforms and saving. Much of the work since has gone into adapting the script runner to that editor. V2 starts from the editor and keeps the script runner as one of its uses.

3.2 What V1 does well, and V2 shall keep

- One file per drawable object, each small and readable, with a header comment the Notes popup shows.
- Controls declared beside the settings and built by the app, used about 1,200 times across the library.
- A single drawing choke point (`drawLine`), and the rule that draw code is separate from the interface.
- Permanent object names, stable layer numbers, and stacking order kept separate from numbering.
- Layer transforms (move and turn without rewriting the shape), and infrastructure as a named account of where discs go.
- `categories.js`: a list builder with no knowledge of manifests or tabs. The best-isolated module in V1.
- Gallery scripts as a playpen, never promoted as-is; a local Node service with its path-safety module.

3.3 Problems

Each problem below is stated with the evidence found in V1. Section 9 and Appendix A show how V2 resolves each one.

No.	Problem	Evidence in V1
P-1	Geometry exists only while it is being painted.	`drawLine` is both the choke point and the paint call. Bounds are collected as a side effect; `infrastructure()` repeats what `draw()` knows; thumbnails borrow the main canvas.
P-2	Draw objects are shared singletons holding live state.	Draw, Figures and Patterns share one set of settings. Figures works around it with `Object.create`, under which any field an instance forgets is silently read from the shared original.
P-3	Settings, handle copies and derived values share one object, and all are saved.	`params` holds `pt1`, a `points[]` mirror indexed by position, and a computed `midpoint`. A saved file can disagree with itself.
P-4	`update()` guesses what was dragged.	Parabola compares midpoints to decide; array entries must be changed in place because the interactor holds references; `if (incoming.lineWidth)` ignores a value of 0.
P-5	Handles are not declared.	Weld code finds handles by comparing coordinates to a millionth of a pixel (`findHandleIndex`); `cornerPartners()` exists to compensate.
P-6	Welds keep copies in step after every move.	The propagation walk with four special cases (729 lines before the split). Both bugs found in Phase 8 use were here.
P-7	No undo.	Delete Layer warns that it cannot be undone. The October triage deferred undo (G5).
P-8	Three kinds of state share one object.	`uiState.figures.tabs[id]` holds the figure, live instances, the saved snapshot, mode and selection. Choosing a layer reorders the stack and needed a special rule not to mark the figure changed.
P-9	Tab switching follows a decision order of flags.	`needsUpdate`, then `init(true)`, else `saved` then `restore`, else `init(false)`. No tab's `save()` is ever called.
P-10	Manifests are stored copies of facts that live in the folders.	Start-up rebuilds, missing-file checks, the Home alert bar, the title-overwrite bug. Figures loads its own manifests, bypassing `ManifestManager`.
P-11	The file and Node boundaries are bypassed.	15 direct `fetch()` calls outside `fileLayer`; 10 UI modules call `nodeDispatch` with paths they build.
P-12	Scripts share global state.	Scripts read `ctx`, `drawState` and `window` names; two scripts fail after about 100 earlier runs.
P-13	Adding a script or object is not immediate.	A new script appears after a restart or Rebuild; a new Draw object needs an edit to `drawRegistry.js`. `boxes.js` is loaded but silently absent.

P-14	Controls have drifted into overlapping conventions.	Separate `params` and `controls` blocks; widget names instead of types; `color` and `colorPicker`; three keys for the widget name; widgets write settings directly; a router guesses the tab.
P-15	No real units and no true-scale output.	Coordinates are canvas pixels. Print aids (Phase 9) are not built.
P-16	Vocabulary and menus are inconsistent.	"Overlay" has three meanings. Each tab builds its menus differently (eight `*MenuCmds.js` files).
P-17	Object metadata is decorative.	Seven fields read by nothing; `version` drives nothing; no migration exists for saved data.
P-18	Parametric and non-parametric drawings use separate code.	`drawParametrics.js` stands apart; `createNodes` (even spacing, bias, jitter) serves ellipses only.
P-19	Adapting a foreign script is manual.	Lifting constants into controls is done by hand per script, producing `scriptInfo`, a second format that still needs promotion.

4.0 CONCEPT OF THE NEW SYSTEM

4.1 The central idea

Underneath every line Sketchpad draws is an ordered array of points, and the lines are a rule for joining entries of that array. V2 is built around that fact. A **path** is an array of points with a flag saying whether it closes on itself. Every drawing is made in four steps: **make** one or more paths, **shape** them, **connect** them into stitches, and **find** the named points on them. Parametric curves are simply one kind of maker, so the distinction between parametric and non-parametric drawings disappears below the maker (P-18).

Three things are kept strictly apart:

1. **Kernel**: the geometry code. Values in, values out. No canvas, no page, no state.
2. **Generators**: one file per drawable object or script, declaring its settings and returning geometry. Generators replace both the Draw registry and the script formats.
3. **Documents**: plain JSON describing what was made from generators: layers, the point table, welds, transforms. No code and no derived values.

Drawing, hit-testing, thumbnails, welds and printing all read geometry as data (P-1). Every change to a document is a named command (P-7). The folders are the truth, and indexes are worked out from them (P-10, P-13).

4.2 Decisions recorded with the owner

No.	Decision
D-1	V2 is a rebuild from scratch, done on its own branch. V1 remains usable throughout.
D-2	The iPad is used for review only. No touch authoring, browser storage or offline editing is required.
D-3	A weld is a shared point number held in a point table. The owner's rule of 23 October 2026, decision 8.
D-4	V2 need not reproduce V1 output exactly. The approved V2 output becomes the baseline.
D-5	The Draw registry objects are redone completely.
D-6	Adding a script or object by saving a file, with no other step, is a defining feature.
D-7	Draw functions stay separate from the interface. V2 sharpens this into kernel and render.

4.3 Assumptions (agreed in discussion, open to change)

No.	Assumption
A-1	The six tabs remain, over one common engine (the "one workspace" alternative was withdrawn).
A-2	Plain JavaScript modules, no framework, no build step. JSDoc type comments are optional.
A-3	Undo by whole-document snapshots, not paired apply and undo functions.
A-4	Explicit redraws after each command, not a publish and subscribe store (the October triage rejected an event bus, D2).
A-5	Two Parabola corner handles remain independent until the owner welds them.
A-6	Playpen scripts may still paint with a raw canvas context, inside the script host.

4.4 Earlier answers reconsidered

In October, before Figures existed, the owner ruled out several things that V2 now includes. Those answers protected working code; a rebuild changes that premise. They are listed so they are reconsidered deliberately, not silently reversed.

October item	Answer then	V2
G5 Undo	Deferred	Included (FR-6.6)
Spec decision 14	No schema versioning	Versioned format with migrations (FR-6.5.6)
Spec decision 2	Canvas pixels only	Real units for output (FR-6.11)
G1 SVG export	Eliminated	SVG and PDF output (FR-6.11)
B12, B16, B22 state machine, declarative engine, declarative templates	Deferred	Documents and commands cover their intent

5.0 ORGANIZATION AND MODULARIZATION

5.1 Principle of division

V1 divided its code mostly by *mechanism*: how a thing is done (an HTTP read, a Node request, building the page). V2 divides by *purpose*: what a thing is for (the catalog, the document, storage). A module is defined by the one kind of data it owns, the modules it may call, and the modules it must never call. A boundary that cannot be checked by a script will be bypassed, as `fileLayer` and `nodeLayer` were (P-11); every V2 boundary is therefore checkable (FR-6.1.4).

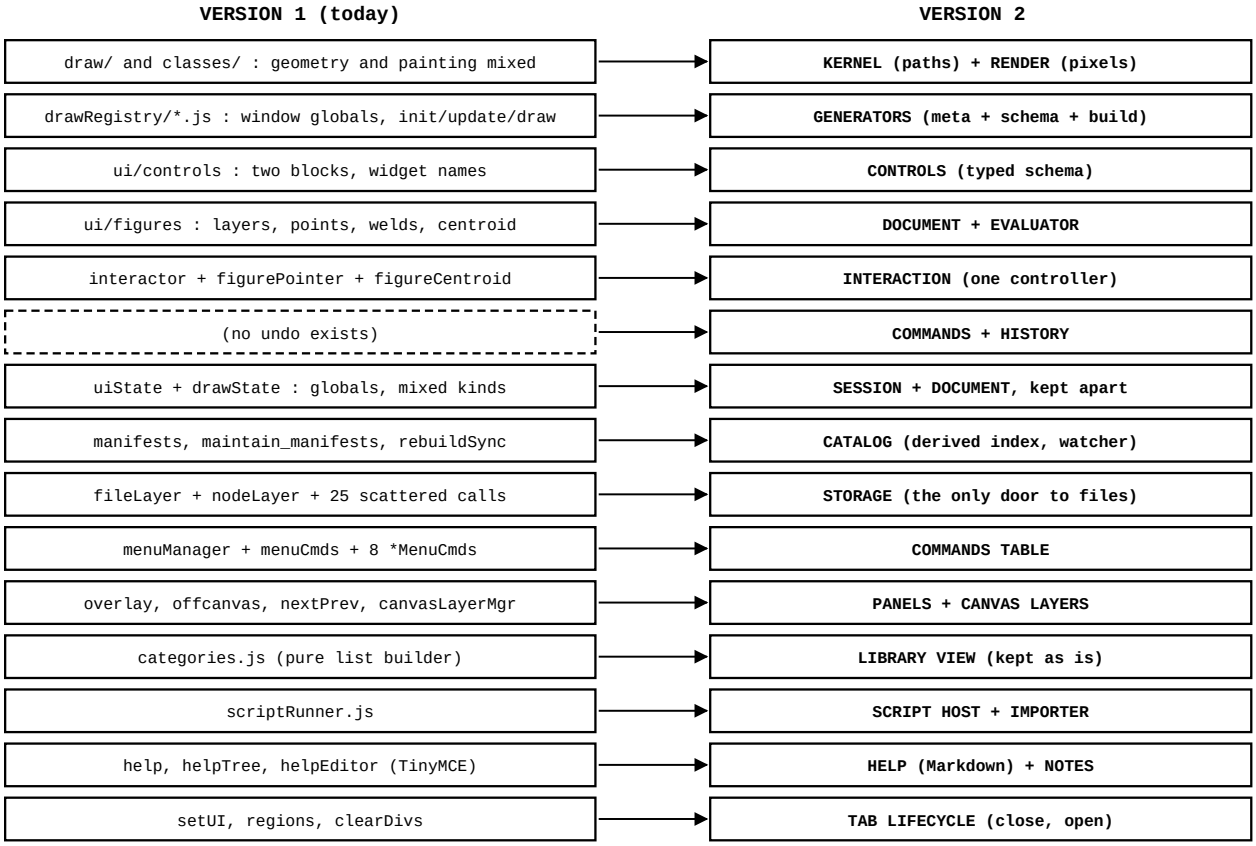


Figure 1. Version 1 components and their Version 2 replacements

5.2 Layers of the system

Figure 2 is a layered structure chart. A module may call modules in the bands below it and never above. The kernel calls nothing. Only `main.js` may touch the browser's `window` object. The Node service is a separate process, reached only through Storage and Catalog.

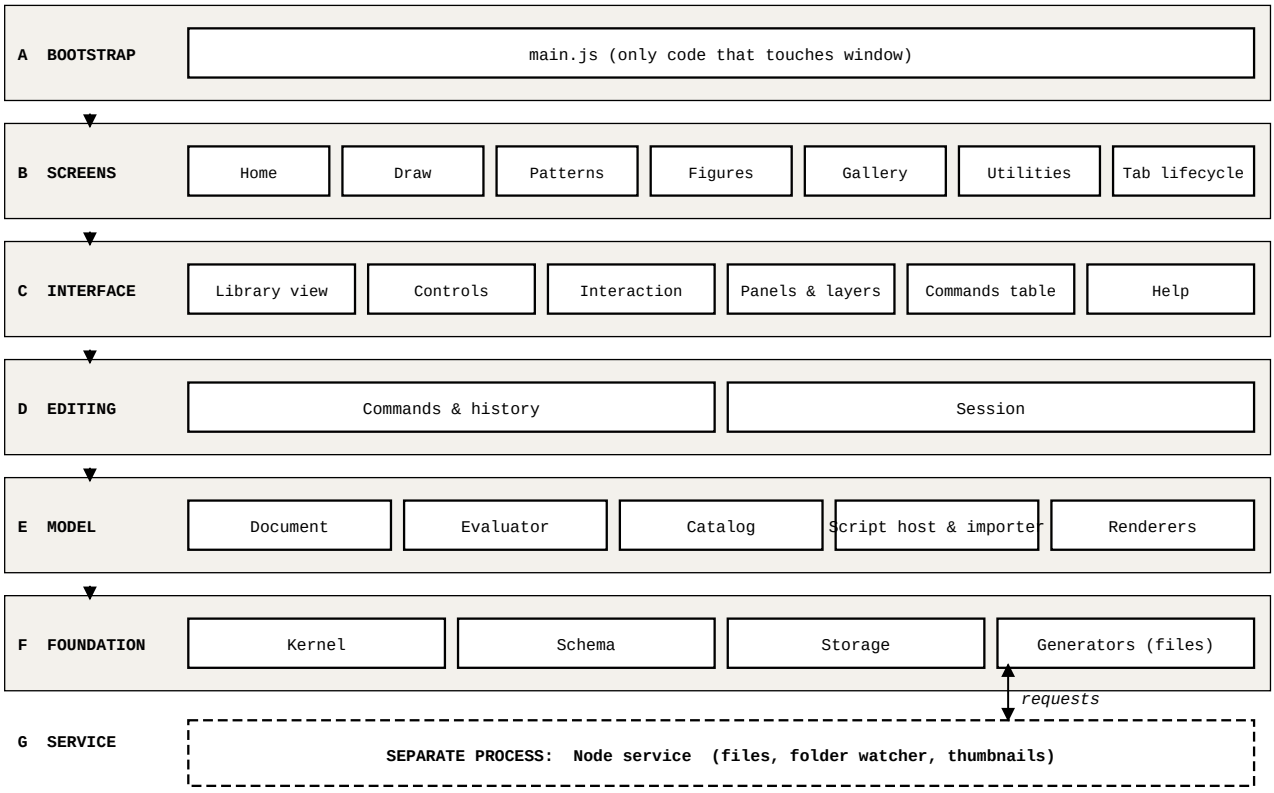


Figure 2. Structure chart: the bands of V2 and the direction of calls

5.3 Draw separation, sharpened

The owner's rule "UI owns the definition; Draw owns the implementation" held in one direction in V1: nothing in `draw/` or `classes/` imports from `ui/`. It blurred in two ways. First, "draw" mixed geometry with painting: `drawState.js` creates the canvas and holds global `ctx`, `transform`, `bounds` and `final`, and four draw files touch the page. Second, the interface paints too: discs, weld marks, handles and the script runner call `ctx` from `ui/`. V2 keeps the rule and moves its boundary one step earlier than `drawLine`, as Figure 3 shows.

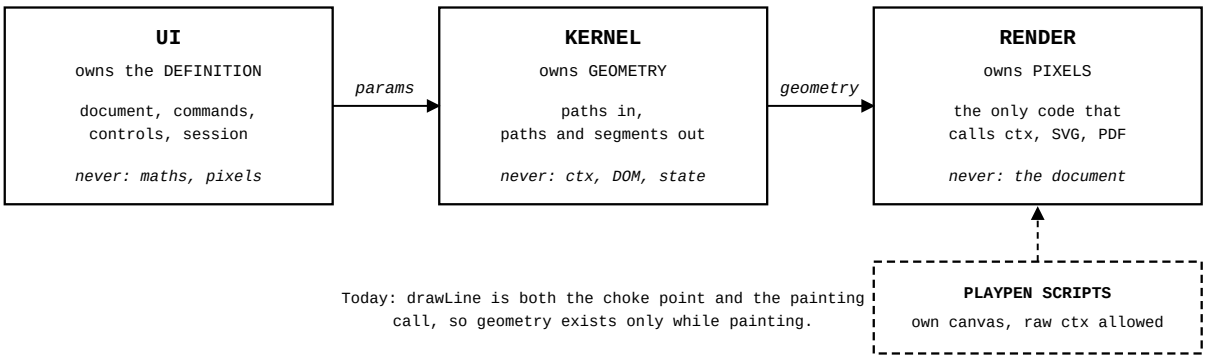


Figure 3. The three layers: definition, geometry, pixels

5.4 Module definitions

Id	Module	Owns	May call	Must never
M-1	Kernel	Paths; makers, shapers, connectors, finders; transforms; bounds	nothing	touch ctx, the page, window or any state
M-2	Generators (files)	One drawable object or script each: meta, settings schema, build	the kernel, passed in as an argument	import draw or ui code; keep state

M-3	Schema	Setting types, the generator format, the document format, migrations, name rules	nothing	touch files
M-4	Document	The open document as plain data	Schema	be changed except through Commands
M-5	Evaluator	Turning a document into geometry, layer by layer in dependency order	Kernel, Generators, Schema	change the document
M-6	Commands and History	Every document change; snapshots; undo and redo; the saved position	Document, Evaluator, Schema	touch the page
M-7	Session	View state per tab and per open document: selection, mode, zoom, open subtabs	nothing	be saved in a document
M-8	Renderers	Pixels and output files: canvas, SVG, PDF, nail list; the passes for discs, welds and handles	Kernel	read or change the document
M-9	Interaction	One mouse and keyboard controller: screen to document coordinates, hit testing, gestures	Session, Commands, Renderers (hit data)	change the document directly
M-10	Controls	Building widgets from a settings schema	Schema, Commands	write settings directly
M-11	Commands table	Every menu item and button: label, enabled rule, action	Commands, Storage (via Catalog), Session	contain drawing code
M-12	Panels and canvas layers	Page side areas (panels) and drawing surfaces (canvas layers)	nothing	know about documents
M-13	Library view	Category lists and thumbnails; V1's `categories.js` rule kept	nothing; it is handed a descriptor	know about manifests or tabs
M-14	Catalog	The index of every library item, built from the folders and per-item metadata	Storage	store copies of facts held elsewhere
M-15	Storage	The only door to files and the Node service	Node service	be bypassed (no other module fetches)
M-16	Script host	Running scripts in an isolated frame with a fresh canvas; the legacy adapter; the recorder	Storage (to load code), Renderers	share state between runs
M-17	Importer	Analysing a script and proposing the ingestion levels	Script host, Schema, Storage	rewrite a file without approval
M-18	Help and notes	Markdown help pages; header notes	Storage	load an editor until asked
M-19	Tab lifecycle	Closing one tab and opening another	Session, Panels	keep flags such as needsUpdate
M-20	Bootstrap	`main.js`: creates the single `app` object and starts the screens	all	(the only module that touches `window`)
M-21	Node service	Files, the folder watcher, thumbnails, path safety	the file system	build page content

5.5 Folder layout

V2 lives in `app2/` beside V1 on the version-2 branch. The library folders (`patterns/`, `gallery/`, `utilities/`, `figures/`) stay where they are and are read by both, so V1 keeps working and nothing is copied. New generators live in `generators/`.

```
app2/
  main.js                   M-20  bootstrap; the only window access
  kernel/                   M-1   paths, makers, shapers, connectors, finders, transform
  schema/                   M-3   types, generator format, document format, migrate, names
  model/                    M-4..M-7, M-14, M-15
    document.js  evaluator.js  commands.js  history.js  session.js  catalog.js  storage.js
  render/                   M-8   canvas.js  svg.js  pdf.js  nails.js  passes.js
  ui/                       M-9..M-13, M-18, M-19
    tabs/ home.js draw.js patterns.js figures.js gallery.js utilities.js
    library.js  controls/  interaction.js  commandsTable.js  panels.js  help.js  lifecycle.js
  host/                     M-16, M-17   scriptHost.js  legacyAdapter.js  recorder.js  importer/
  node/                     M-21  server.mjs  storage.mjs  catalog.mjs  watcher.mjs  thumbnails.mjs  safePaths.mjs
  vendor/                         acorn.js (if approved, O-4)  d3-delaunay.js
  tests/                          kernel, evaluator, commands, storage (Node); screens (Playwright)
generators/                 one file per drawable object (replaces drawRegistry/)
patterns/ gallery/ utilities/ figures/     unchanged locations, shared with V1
```

5.6 Screen layout

The six tabs remain (A-1). Within a tab, the screen has the regions shown in Figure 4. The library list, the layer strip, the canvas and the inspector are the same components in every tab that uses them; a tab chooses which to show.

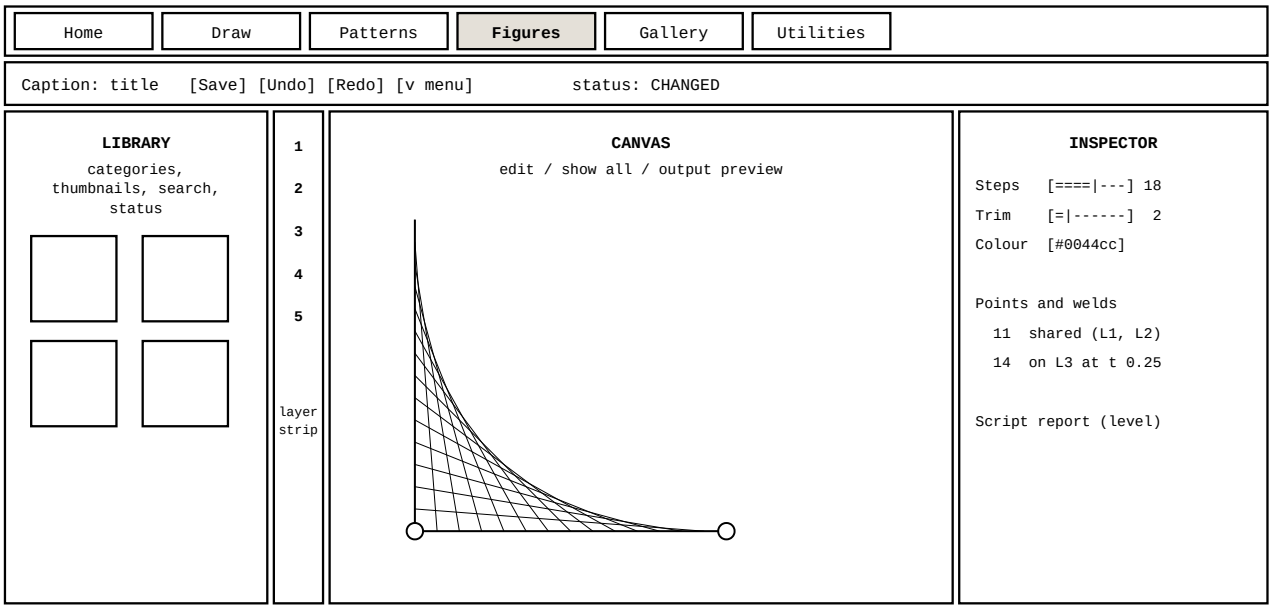


Figure 4. Screen regions, shown on the Figures tab

6.0 FUNCTIONAL REQUIREMENTS

6.1 General

FR-6.1.1	V2 shall be built on its own branch in `app2/`, leaving V1 usable until cut-over (D-1).
FR-6.1.2	V2 shall be plain JavaScript modules served without a build step (A-2).
FR-6.1.3	Every source file shall begin with a header comment stating what it does; the Notes popup shall show it.
FR-6.1.4	An automatic check shall verify the call rules of Section 5.4 (which module may import which) and fail the build on a violation. It shall run with the other checks.
FR-6.1.5	Only `main.js` shall read or write the browser's `window` object.

6.2 Kernel: paths

A path is an ordered array of points with a flag saying whether it closes on itself. Figure 5 shows an open path and a closed path. When a path is closed, an index past the last point wraps to the first.

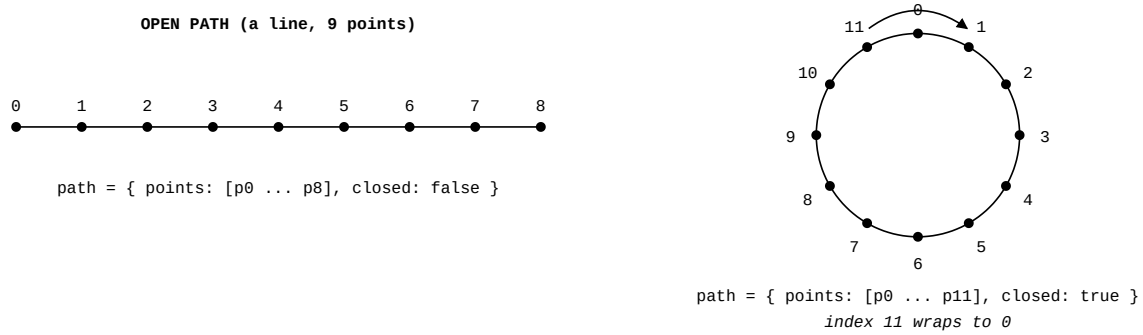


Figure 5. Paths: the array of points underneath every drawing

Kernel operations fall into four families. Makers build a path from a rule. Figure 6 shows that a line, a circle and a parametric curve differ only in their rule: each is sampled into a path the same way, which is what removes the parametric and non-parametric divide (P-18).

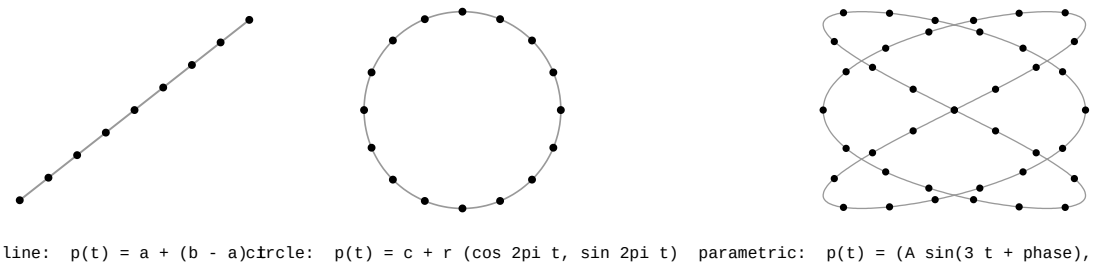


Figure 6. Makers: three rules, one kind of result

Shapers turn a path into another path. They are V1's `createNodes` pipeline (sample finely, space by arc length, bias, jitter) taken apart so that each step works on any path, not only an ellipse. Figure 7 shows the principal shapers.

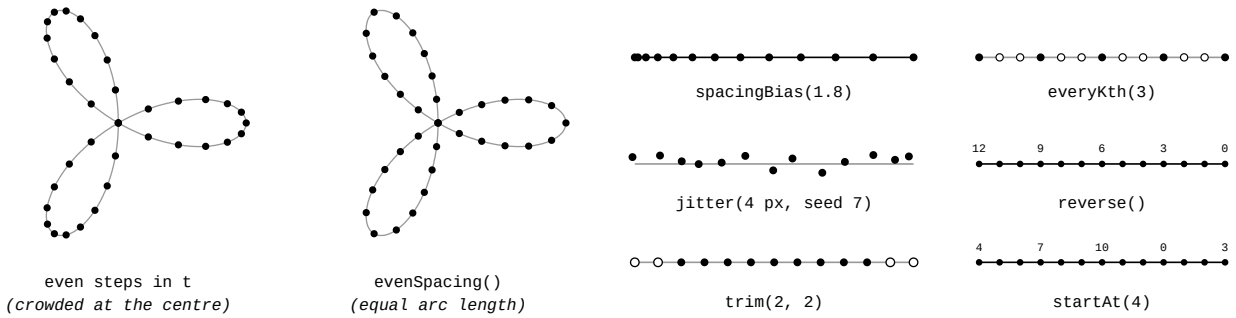


Figure 7. Shapers: path in, path out

Connectors join the points of one or more paths into stitch lines. Each of V1's string-art objects is one connector applied to one or two paths, as Figure 8 shows.

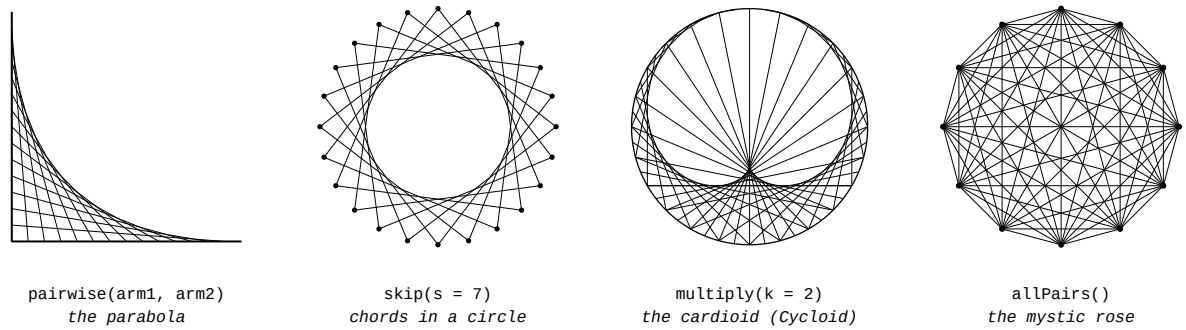


Figure 8. Connectors: four rules that produce four familiar objects

FR-6.2.1	The kernel shall represent a path as `{ points, closed }` (Section 8).
FR-6.2.2	The kernel shall provide makers at least for: line, polygon, circle, ellipse, arc, spiral, and parametric curve `p(t)` over a range of `t`.
FR-6.2.3	The kernel shall provide shapers at least for: move, rotate, scale; even spacing by arc length; spacing bias; jitter; trim; reverse; start at index; every k-th point; split; join; mirror.
FR-6.2.4	Jitter shall take a seed, so the same settings give the same picture on every redraw.
FR-6.2.5	The kernel shall provide connectors at least for: pairwise between two paths, skip by s, multiply by k, all pairs, ring, and connection among several paths (Linked Circles).
FR-6.2.6	When two paths of different length are connected, the connector shall either resample to match, or stop with a message naming both lengths and the missing index, as the owner asked in October.
FR-6.2.7	The kernel shall provide finders at least for: ends, midpoints, extremes, lobe tips, cusps and crossings. A finder shall name a point by feature (for example "tip of lobe 3"), not by array index.
FR-6.2.8	A curve shall have two samplings: a fine one for display and an n-point one for nails and stitches.
FR-6.2.9	No kernel function shall read or write anything other than its arguments and its return value.

6.3 Generators

A generator is one file that describes one drawable object or one script. It replaces both the V1 registry object and the V1 `scriptInfo` script. Listing 1 shows a complete generator for the Parabola. Its corner is two handles, independent until welded (A-5).

```
/* generators/parabola.js  -- two arms stitched pairwise: the curve-stitch parabola. */
export const meta = {
  id: "parabola", version: 1, title: "Parabola",
  tags: ["stitch"], notes: "Two arms stitched at equal steps."
};

export const params = {                // one entry per setting; controls come from this
  a:      { type: "point", handle: true, default: [100, 100] },
  b1:     { type: "point", handle: true, default: [100, 300] },   // arm 1 corner end
  b2:     { type: "point", handle: true, default: [100, 300] },   // arm 2 corner end
  c:      { type: "point", handle: true, default: [300, 300] },
  steps:  { type: "int",   default: 18, min: 3, max: 64, label: "Steps" },
  trim:   { type: "int",   default: 0,  min: 0, max: 15, label: "Trim" },
  color:  { type: "color", default: "#0044cc" }
};

export function build(p, k) {          // k is the kernel, passed in
  const arm1 = k.trim(k.line(p.a,  p.b1, p.steps), p.trim, 0);
  const arm2 = k.line(p.b2, p.c,  p.steps);
  return { paths: { arm1, arm2 },        // infrastructure: where discs may go
           stitches: k.pairwise(arm1, arm2),
           style: { color: p.color } };
}
```

Listing 1. A complete generator. There is no init, update or draw, and nothing is stored on the object.

FR-6.3.1	A generator file shall export `meta`, `params` and one of `build(p, k)` (geometry) or `run(api)` (a painting script). It may export `drag(name, position, p)` and `migrate(oldVersion, p)`.
FR-6.3.2	`meta.id` shall be permanent: it is what saved documents refer to. Renaming the file shall have no effect on documents. Two files with the same id shall both be reported and neither loaded.
FR-6.3.3	`meta.version` shall be a whole number. When it rises, `migrate` shall convert settings saved under earlier versions.
FR-6.3.4	`build` shall return, in one result, everything needed by every reader: paths (infrastructure), stitches, named points, bounds and style. There shall be no second method that must agree with it.
FR-6.3.5	A setting marked `handle: true` shall be draggable on the canvas. Dragging writes that setting through a command and `build` runs again.
FR-6.3.6	Where a handle's behaviour is not "move this point", the generator's `drag` shall return the new settings (for example the Circular Parabola Multi handle that separates and joins copies).
FR-6.3.7	A generator may declare a handle that moves others (for example a midpoint that moves a whole arm) with `moves: [ ... ]`.
FR-6.3.8	Each generator file shall be checked against the format when loaded. A failure shall show a badge with the reason on that item only; the rest of the library shall load.
FR-6.3.9	The 14 V1 Draw objects shall be rewritten as generators (D-5). Exact reproduction of V1 output is not required (D-4). Before each is written, a one-page feature list (controls, handles and what each does, infrastructure, special behaviour) shall be drafted from V1 and corrected by the owner, so nothing distinctive is lost.

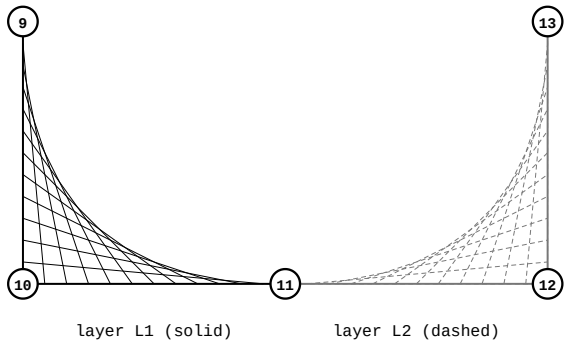
6.4 Settings schema and controls

FR-6.4.1	Each setting shall be declared once, with its type, default and limits, in `params`. There shall be no separate `controls` block (P-14).
FR-6.4.2	The types shall be: `int`, `number`, `angle`, `color`, `choice`, `bool`, `text`, `point`, `points` (a list) and `pathRef` (a reference to another layer's path).
FR-6.4.3	Each type shall have a default widget. A setting may name a different widget with `widget:`. Conversion and checking of values shall happen in one place, by type, not in each widget.
FR-6.4.4	A widget shall not write a setting. It shall send one `SetParam` command. A whole slider movement shall be one undo step.
FR-6.4.5	A setting may carry `section:` (grouping, shown as an accordion) and `showIf: { setting: value }` (conditional display).
FR-6.4.6	Number settings shall show a number box beside the slider for exact entry, and every setting shall offer reset to its default.
FR-6.4.7	Buttons are actions, not settings, and belong to the commands table (6.9). Static text belongs to notes or help.
FR-6.4.8	A converter shall turn V1 declarations into the schema mechanically: `range` with whole-number bounds and step 1 to `int`, other `range` to `number` (or `angle` where named so), `color` and `colorPicker` to `color`, `select` and `radio` to `choice`, `checkbox` to `bool`, any leftover `pointPicker` to a handle.

6.5 Documents, the point table and welds

A document holds layers and a point table. A layer's handles name point numbers; they do not hold coordinates. A table entry either holds a position (a free point) or refers to a place on another layer's path (an attached point). These two kinds of entry give exactly two kinds of weld.

Shared point. Two handles name the same free point. Figure 9 shows two Parabola layers sharing point 11. The point is stored once; dragging it moves both layers. This is the owner's rule D-3: the table entry is the point, so nothing is copied and nothing can drift (P-6).

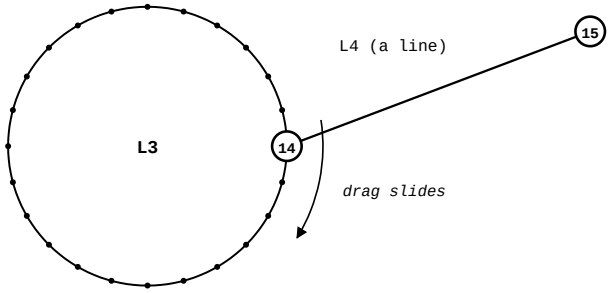



```
points                      layers
 9: (60, 30)                L1: parabola
10: (60, 190)                 line1_pt1  -> 9
11: (220, 190)                line1_pt2  -> 10
12: (380, 190)                line2_pt1  -> 10
13: (380, 30)                 line2_pt2  -> 11
                            L2: parabola
                              line1_pt1  -> 11
                              line1_pt2  -> 12
                              line2_pt1  -> 12
                              line2_pt2  -> 13
```

Point 11 is stored once. Both layers name it.
Drag it and both redraw. Nothing is copied.

Figure 9. A shared point: two layers name point 11

Attachment. A handle names a point that lies on another layer's path, at a fraction `t` along it or on a named feature. It is one-way: the path decides where the point is and the handle follows. Dragging the attached handle slides it along the path. Figure 10 shows a line attached to a circle. Anchored discs, handles dropped mid-arm and scaffold all work this way.



L4 follows L3. L3 never follows L4.

```
points
14: { on: "L3", t: 0.25 }    attached
15: (420, 50)                free

L4: line
  pt1 -> 14
  pt2 -> 15
```

Figure 10. An attachment: point 14 lies on layer L3's path

Because attachments are one-way, layers can be evaluated in order: each after every layer it is attached to. A loop of attachments cannot be evaluated and is refused at the moment the weld is made, with a message (Figure 11). The V1 propagation walk, its snapshots and deltas, its four special cases and `cornerPartners()` are not needed.

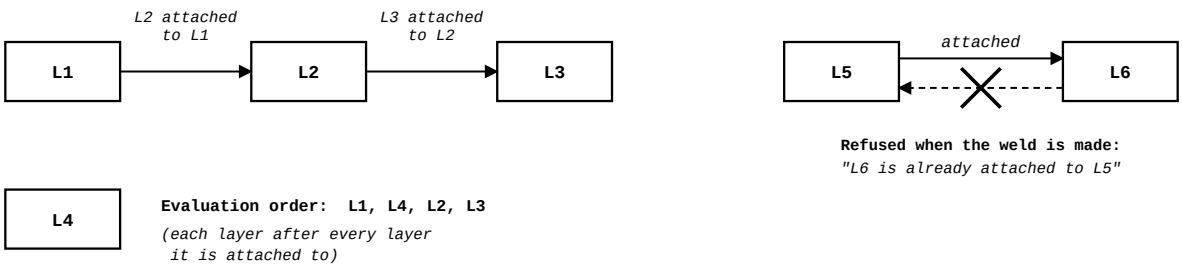


Figure 11. Evaluation order, and a loop refused at weld time

FR-6.5.1	A document shall hold a point table; each entry shall be either a free point `{x, y}` or an attached point `{on, t}` or `{on, feature}`.
FR-6.5.2	Point numbers shall be permanent and never reused (V1's `nextPoint` rule). Disc numbers, point numbers and nail numbers shall be the same numbers.
FR-6.5.3	Welding two handles shall make them name one free point. Unwelding shall copy the point under a new number and point one handle at the copy.
FR-6.5.4	Dropping a handle within the snap radius of another handle shall weld them on release, never while passing over (V1 Phase 8 next pass behaviour kept).
FR-6.5.5	Attaching shall be refused, with a message naming the layers, if it would create a loop.
FR-6.5.6	A document shall carry `format` (a whole number). The loader shall apply one upgrade step per format number, so old files always open. Unknown generator ids shall open as visible placeholders with their data kept.
FR-6.5.7	A document shall contain no derived values and no code. Anything computable shall be computed on load.
FR-6.5.8	A document shall record the units and scale chosen for output (O-1). Documents imported without a scale shall show "scale not established".
FR-6.5.9	When a path's point count changes, an attachment by node shall move to the nearest node; one that cannot be placed shall be reported, never silently moved to whatever point now has its old index.
FR-6.5.10	A layer may be a reference to another saved document (nesting). A reference shall follow edits to that document; a copy shall not. Break apart shall expand a nested document or turn a generator's stitches into layers.
FR-6.5.11	A layer shall have a role: `stitch`, `guide`, `scaffold` or `image`. Guides and scaffold shall never print. Scaffold is one-way by construction: other layers attach to it.

6.6 Commands and undo

Every change to a document goes through a named command. Before a command runs, the whole document is copied onto the history (A-3). Undo restores the previous copy; redo moves forward. A snapshot cannot be wrong in the way a hand-written reverse function can: it puts back exactly what was there, partners included.

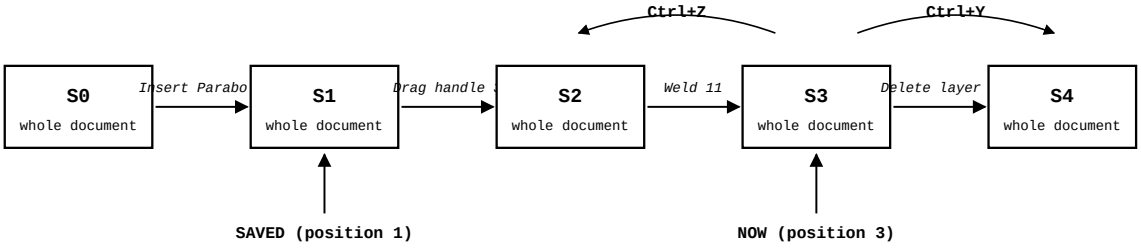


Figure 12. Snapshot history, the current position and the saved position

FR-6.6.1	Nothing shall change a document except a command. Commands shall at least cover: insert layer, delete layer, set setting, move point, weld, unweld, attach, add disc, delete disc, move or turn layer, reorder layers, set layer colour, set output inclusion, break apart, revert to saved.
FR-6.6.2	One gesture shall be one undo step: a handle drag is recorded on release; a slider when movement starts, the whole movement one step; consecutive arrow-key nudges one step until a pause or a change of selection.
FR-6.6.3	Undo shall not cover session changes (selection, mode, Show All, zoom) or file operations (Save, Save As, Archive). File operations shall continue to ask first where V1 asks.
FR-6.6.4	"Unsaved changes" shall mean the history position differs from the position at the last save. Undoing back to that position shall make the document clean.
FR-6.6.5	History shall last while the document is open, including across tab switches, and shall not be saved in the file.
FR-6.6.6	Ctrl+Z shall undo and Ctrl+Y shall redo, except while a text or number field has focus, where the field's own undo applies.
FR-6.6.7	Delete Layer shall no longer warn that it cannot be undone. Reset shall become "revert to saved", an ordinary command that can itself be undone.

FR-6.6.8	Every command shall end by calling one `refresh()`, which redraws the open tab's regions in a fixed order (A-4).

6.7 Session state and tab switching

V1 keeps three kinds of state in one object (P-8). V2 keeps them apart.

Kind	Examples	Saved in the file	Undoable	Survives a tab switch
Document	layers, settings, points, welds, transforms, colours, stacking, output inclusion	yes	yes	yes
Session	open subtabs, active layer, mode, Show All, hidden while editing, selected point, zoom	no	no	yes
Interaction	a drag in progress, hover	no	no	no (ends with the gesture)

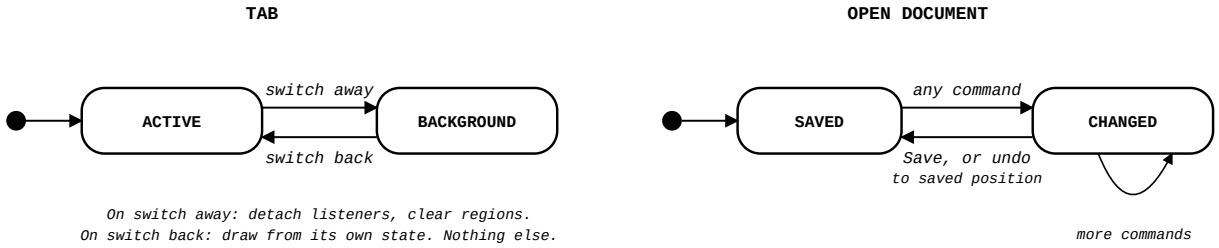


Figure 13. State transitions of a tab and of an open document

FR-6.7.1	Document, session and interaction state shall be held in separate objects. Only the document shall be saved or undone.
FR-6.7.2	The active layer shall be session state. Choosing a layer shall not mark the document changed.
FR-6.7.3	Choosing a layer and reordering layers shall be separate actions. V1's bring-to-front on select may remain as an option (O-6).
FR-6.7.4	"Hidden while editing" shall be session state; "left out of output" shall be document state. Show All shall reveal hidden layers as in V1.
FR-6.7.5	Switching tabs shall close the old tab (detach its listeners, clear its regions) and open the new one, which draws from its own state. There shall be no `needsUpdate` or `saved` flags.
FR-6.7.6	A tab shall re-read the catalog when it opens.

6.8 Catalog and storage

FR-6.8.1	A folder shall be a category. `patterns/builds/myStar.js` is in Patterns, category builds. There shall be no registration step and no hand-maintained list.
FR-6.8.2	An item's title, tags and notes shall come from its file (`meta`, or the header comment for legacy scripts). Its status shall be kept in a small metadata file beside it. Figures carry their title inside the document.
FR-6.8.3	The catalog index shall be built from the folders, never stored as the authority. If curation needs order or hidden entries, these shall be fields in the metadata file (O-5).
FR-6.8.4	Thumbnails shall be a cache: made when missing or out of date, deletable at any time.
FR-6.8.5	Storage shall be the only module that reads files or calls the Node service. It shall offer named operations (open, list, save, save as, archive, write thumbnail, open in editor), never paths or request names to its callers.
FR-6.8.6	Saving shall be atomic for the document file. A thumbnail or index failure shall not turn a successful save into "not saved"; the index shall be repairable from the folders.
FR-6.8.7	One name check shall apply everywhere, rejecting Windows reserved names (CON, PRN, AUX, NUL, COM1 to COM9, LPT1 to LPT9), trailing dots and spaces, and names differing only by letter case from an existing name.
FR-6.8.8	V1's path-safety rules (`safePaths.mjs`) and foreign-origin refusal shall be kept.

6.9 Adding scripts and objects while the application runs

This is a defining feature (D-6). Saving a file into a library folder is the whole procedure. Figure 14 shows the sequence.

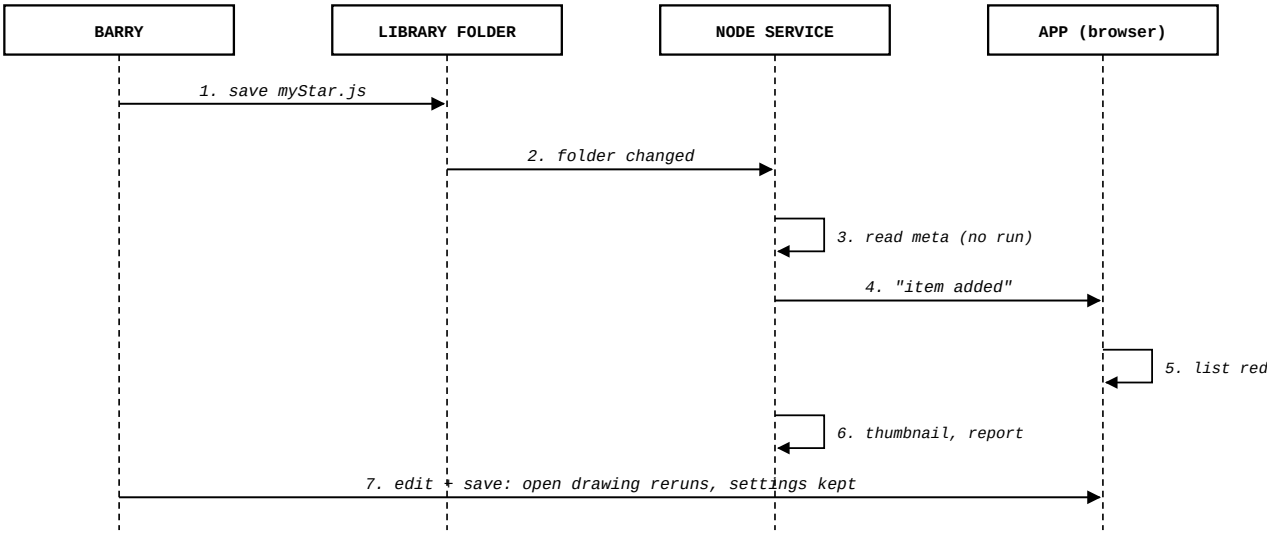


Figure 14. Adding a script: save the file, and it appears

FR-6.9.1	The Node service shall watch the library folders. On a file added, changed or removed, it shall read the item's description without running it, update the index, and notify the open application over a connection kept open for the purpose.
FR-6.9.2	A new item shall appear in its list within two seconds of the file being saved, with no restart and no Rebuild.
FR-6.9.3	When an open item's file changes, it shall rerun with the current settings kept.
FR-6.9.4	Code shall be loaded with the file's change time in its address, so an edited file is never served from the browser cache.
FR-6.9.5	At start-up the service shall scan the folders once, so files added while it was stopped are found. This scan replaces the V1 manifest check.
FR-6.9.6	A generator saved into `generators/` shall appear in the Figures Insert list as a full object, with no other code changed.
FR-6.9.7	"New script" shall write a skeleton native script into a chosen category and open it in VS Code. "Import" shall accept pasted code or a chosen file.
FR-6.9.8	Outside libraries a script needs shall be kept in `vendor/` so everything works without the internet.

6.10 Script host and the ingestion ladder

FR-6.10.1	Every script shall run in an isolated frame with a fresh canvas and fresh state. Nothing shall carry from one run to the next (P-12). Animation frames, timers and listeners shall be stopped when the item closes.
FR-6.10.2	An error shall show as a badge on the item with its message; a run exceeding a time limit shall be stopped. The list shall keep working.
FR-6.10.3	Three kinds of script shall be accepted: **native** (FR-6.3.1), **legacy** (V1 `runPattern` and `scriptInfo`, run through an adapter that supplies the old names inside the frame), and **foreign** (any canvas code, run through the recorder).
FR-6.10.4	The recorder shall supply an object with the canvas drawing methods that records each line and arc as data instead of drawing. A recorded script's output shall be placeable in a figure as a frozen layer.
FR-6.10.5	On first sight of a script, the host shall run it once and save beside it: a report (what it drew, likely stitches, number literals that look like settings, globals touched, internet use, animation, run time) and a thumbnail. Its status shall be `new`.

Adapting a script to built-in controls proceeds by levels (Figure 15). Each level is proposed by the importer, reviewed by the owner, and approved separately; the script works after every level. Level 3, with the owner's minimizing, is promotion: the owner's rule that playpen scripts are never promoted as-is is kept.

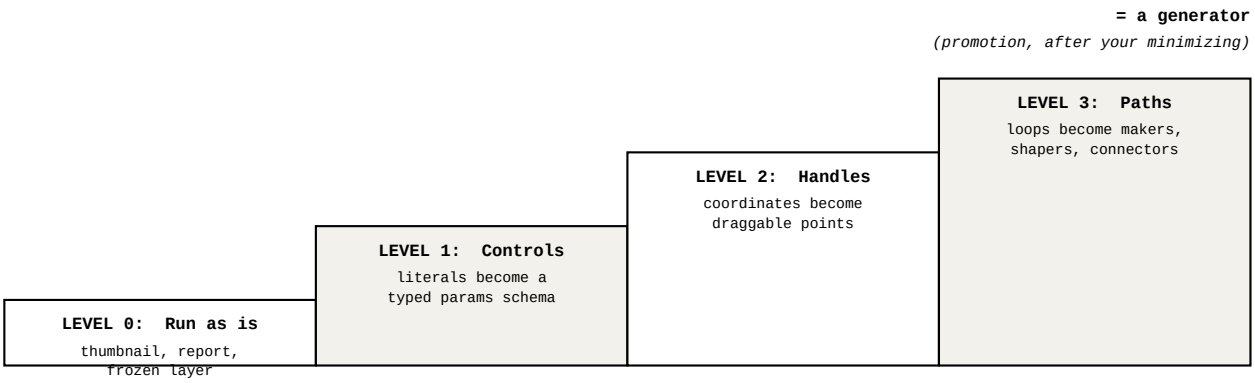


Figure 15. The ingestion ladder

FR-6.10.6	Level 1: the importer shall read the script's structure with a JavaScript parser and propose a typed settings entry for each independent top-level number literal and loop bound, with name, type and range. It shall not propose derived values (`x2 = x1 + xLength`) or constants (`Math.PI / 2`).
FR-6.10.7	Level 2: the importer shall propose point settings with handles from paired names (`x1`/`y1`, `cx`/`cy`) and points built from literals.
FR-6.10.8	Level 3: the importer shall draft a recipe from the recorder's stitch detection (for example "pairwise between two lines, 18 steps").
FR-6.10.9	Each level shall be shown in a review panel: the proposed settings (tick, rename, change ranges), a live preview of the controls, and the code change. Accepting shall rewrite the file and keep the original beside it. Nothing shall be rewritten without approval.
FR-6.10.10	The first test of the importer shall be `patterns/builds/OverlappingStars.js`; its Level 1 proposal shall be compared with the hand adaptation in V1's `OVERLAP.js`.

6.11 Rendering and physical output

FR-6.11.1	Renderers shall be the only code that paints. Each shall take geometry and view options and return pixels or a file.
FR-6.11.2	Final (points only) shall be a renderer option, not a flag read by objects.
FR-6.11.3	The canvas renderer shall draw, in order: layers (with dimming), discs, weld marks, then the interaction layer (handles, selection) on its own canvas.
FR-6.11.4	Output engines shall at least include: SVG; a true-scale PDF with a scale bar and a 100 mm calibration line; and a nail list (number, x and y in the chosen units, weld group).
FR-6.11.5	A weld group (a shared point) shall print as one nail. Coincident points that are not welded shall follow a stated grouping rule shown in an output preview (O-2).
FR-6.11.6	A spacing check shall take nail diameter and clearance (O-3), and highlight and list pairs that are too close. It shall not prevent an experimental drawing from being made.
FR-6.11.7	Tiling across several sheets, with overlap and alignment marks, should be provided if the owner's boards exceed one sheet (O-7).
FR-6.11.8	Thumbnails shall be the renderer drawing small onto its own canvas, never the main canvas.

6.12 Help, notes, menus and the screens

FR-6.12.1	Help pages shall be Markdown files, edited in VS Code. TinyMCE shall be removed.
FR-6.12.2	Every menu item and button shall be an entry in one commands table: `{ id, label, enabledWhen(state), run() }`. A disabled item shall say why.
FR-6.12.3	"Panel" shall mean a page side area and "canvas layer" a drawing surface. "Overlay" shall not be used.
FR-6.12.4	Home, Draw, Patterns, Figures, Gallery and Utilities shall keep their V1 purposes (A-1). Draw shall be a one-layer document; a saved Draw setting shall be a saved document.
FR-6.12.5	Save As shall use one form showing category, title and destination, replacing V1's chain of prompts.

7.0 DATA FLOW

7.1 Context diagram

Figure 16 shows Sketchpad as a single process and everything outside it. The owner gives commands and receives drawings, lists and reports. The file system holds the library and saved documents. VS Code edits files directly; the change reaches Sketchpad through the file system, not through Sketchpad. A foreign script enters by import. The printer receives a true-scale template.

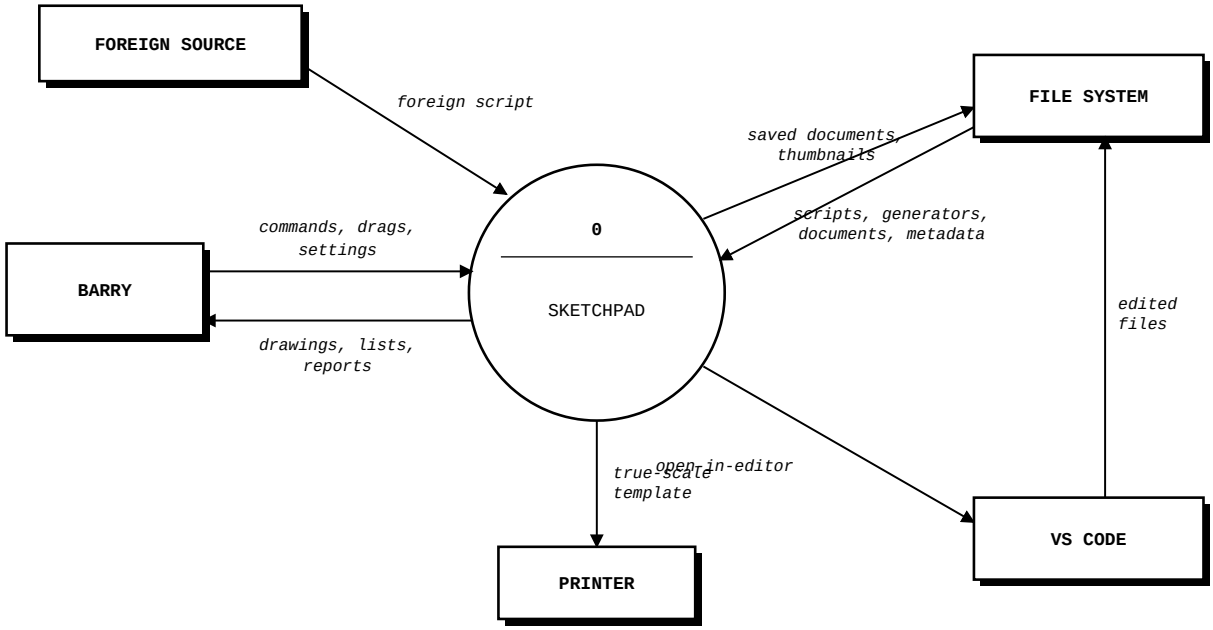


Figure 16. Context diagram (Level 0)

7.2 Level 1: the seven processes

Figure 17 expands process 0. Two flows deserve attention. First, process 2.0 is the only process that writes the open document (D2); every other process reads it. Second, process 1.0 learns of new files from the library folders (D1) directly, by watching them, which is what makes a saved file appear without a Rebuild.

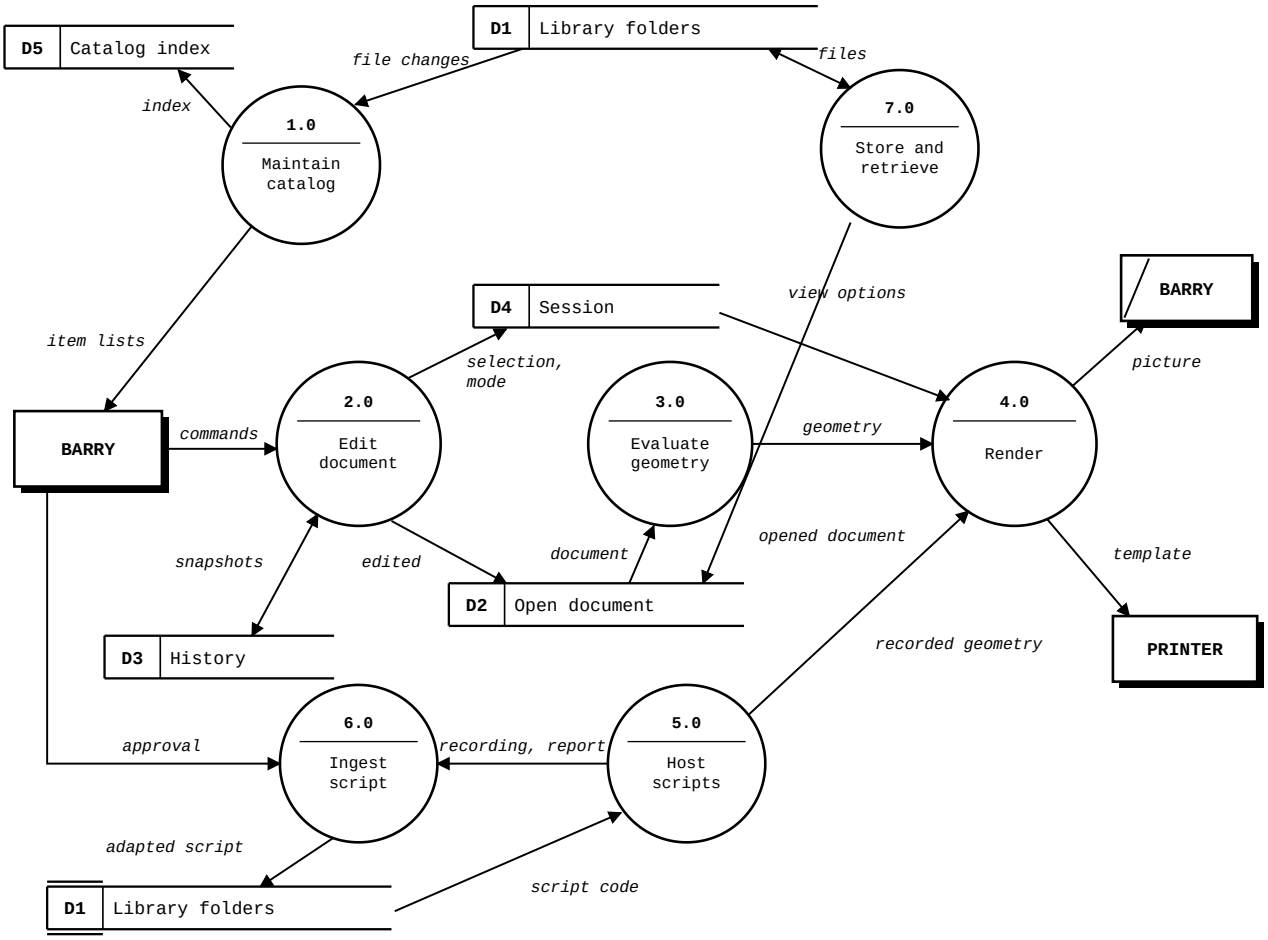


Figure 17. Level 1 data flow diagram

Process	Takes	Gives	Modules
1.0 Maintain catalog	file changes from D1	the index (D5); item lists	Catalog, Node watcher
2.0 Edit document	commands; selection and mode from D4	the edited document (D2); snapshots (D3)	Interaction, Commands, History, Session
3.0 Evaluate geometry	the document (D2)	geometry	Evaluator, Kernel, Generators
4.0 Render	geometry; view options (D4); recorded geometry	picture; template file	Renderers
5.0 Host scripts	script code	recorded geometry or pixels; recording and report	Script host
6.0 Ingest script	recording and report; the owner's approval	adapted script (D1)	Importer
7.0 Store and retrieve	save and open requests	files (D1); opened document (D2)	Storage, Node service

7.3 Level 2: process 2.0, Edit document

A gesture is interpreted against the session (what is selected, which mode), turned into an intent, then a command. The snapshot is taken before the command is applied, so undo can always return to the state before it. The changed document goes on to evaluation. Undo and redo exchange the open document with a snapshot from history.

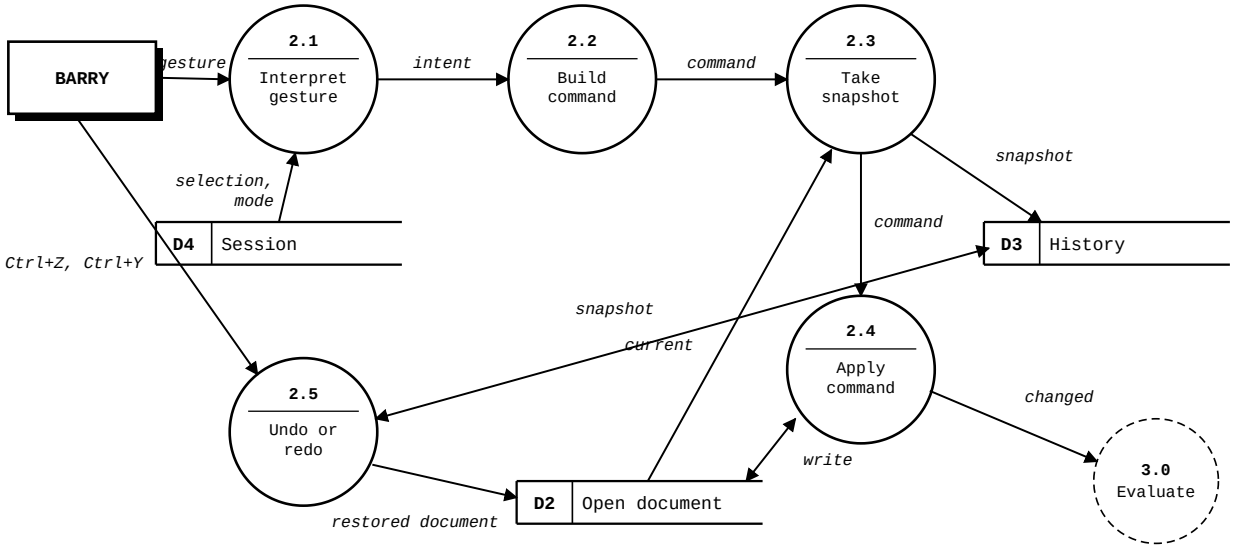


Figure 18. Level 2: Edit document

7.4 Level 2: process 3.0, Evaluate geometry

Layers are taken in dependency order (Figure 11). For each layer, its handles' point numbers are resolved: free points are read from the table, attached points are computed from paths of layers already finished. The generator's recipe makes the paths, shapers shape them, connectors join them into stitches, and finders name the points that later layers and the nail list need. The loop from 3.6 back to 3.2 is how one layer's named points become available to the next.

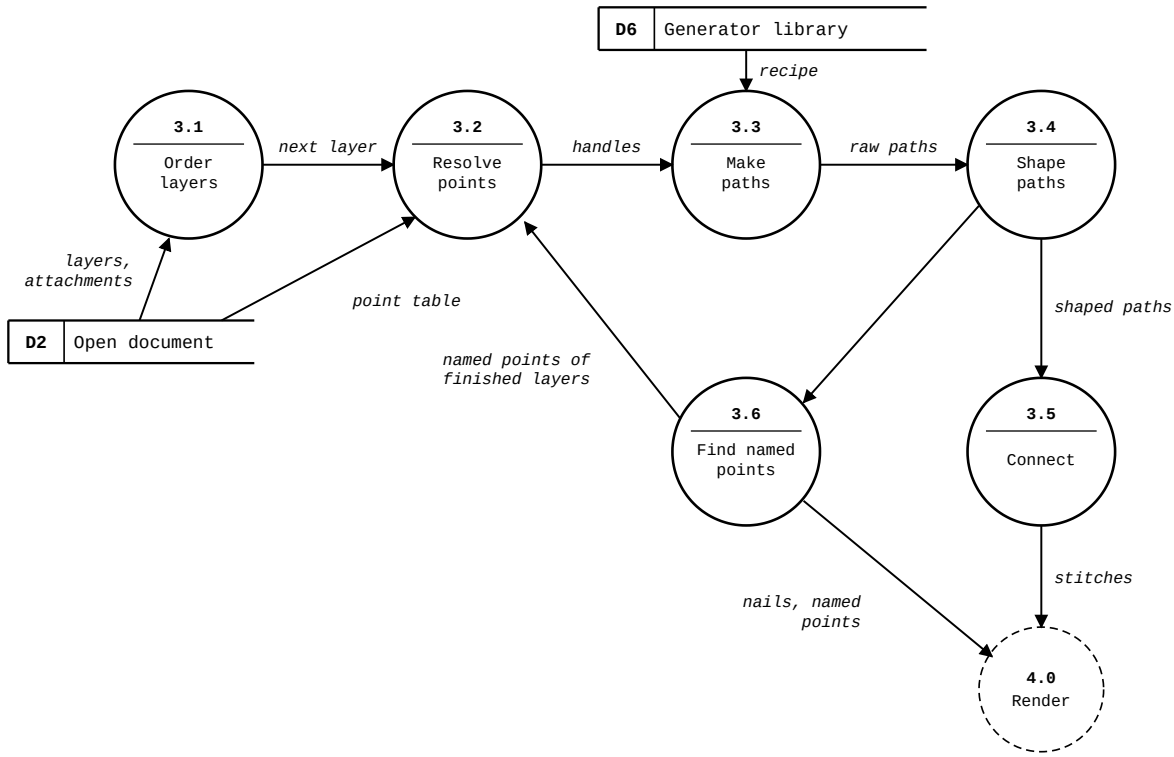


Figure 19. Level 2: Evaluate geometry

8.0 DATA DICTIONARY

DeMarco notation (Section 1.4). Names in capitals are defined elsewhere in this dictionary.

```
DOCUMENT        = format + title + units + (scale) + POINT-TABLE + {LAYER} + layerOrder + OUTPUT-SETTINGS
format          = * whole number; 2 for the first V2 format *
units           = [ "mm" | "in" | "legacy" ]
POINT-TABLE     = {pointNumber + POINT}
pointNumber     = * whole number, permanent, never reused *
POINT           = [ FREE-POINT | ATTACHED-POINT ]
FREE-POINT      = x + y
ATTACHED-POINT  = on + [ t | feature ]
on              = * a layerId *
t               = * fraction along the path, 0 to 1 *
feature         = * a finder's name, e.g. "tip of lobe 3" *
LAYER           = layerId + legacyNumber + [ GENERATOR-USE | DOCUMENT-REF ] + transform + role
                  + identityColor + includeInOutput
GENERATOR-USE   = generatorId + generatorVersion + {setting} + {handleName + pointNumber}
DOCUMENT-REF    = documentPath + [ "reference" | "copy" ]
role            = [ "stitch" | "guide" | "scaffold" | "image" ]
transform       = angle + tx + ty
OUTPUT-SETTINGS = (nailDiameter) + (clearance) + (paper) + (tiling) + (labels)

SESSION         = {tabId + TAB-SESSION}
TAB-SESSION     = {openDocument} + activeLayer + mode + showAll + {hiddenLayer} + (selectedPoint) + zoom + pan
HISTORY         = {SNAPSHOT} + position + savedPosition
SNAPSHOT        = commandName + DOCUMENT

GENERATOR-FILE  = META + PARAMS + [ build | run ] + (drag) + (migrate)
META            = id + version + title + {tag} + notes
PARAMS          = {settingName + SETTING-DEF}
SETTING-DEF     = type + default + (min) + (max) + (step) + (label) + (widget) + (section) + (showIf) + (handle)
type            = [ "int" | "number" | "angle" | "color" | "choice" | "bool" | "text" | "point" | "points" | "pathRef" ]

PATH            = {point} + closed
GEOMETRY        = {pathName + PATH} + {STITCH} + {pointName + point} + bounds + style
STITCH          = point + point

CATALOG-ENTRY   = itemId + kind + category + title + status + sourcePath + (thumbnail) + (report) + (badge)
kind            = [ "generator" | "script" | "legacyScript" | "foreignScript" | "document" | "image" ]
ITEM-METADATA   = status + {tag} + (order) + (hidden)       * the small file beside each item *
SCRIPT-REPORT   = {STITCH-GUESS} + {SETTING-GUESS} + {globalTouched} + usesNetwork + animates + runTime
```

An example document, to show that a figure can be read and understood in VS Code:

```
{ "format": 2, "title": "Two parabolas", "units": "mm", "scale": 0.5,
  "points": { "9": [60, 30], "10": [60, 190], "11": [220, 190],
              "14": { "on": "L3", "t": 0.25 } },
  "layers": [
    { "id": "L1", "n": 1, "use": "parabola@1", "role": "stitch",
      "settings": { "steps": 18, "color": "#0044cc" },
      "handles":  { "a": 9, "b1": 10, "b2": 10, "c": 11 } },
    { "id": "L2", "n": 2, "doc": "figures/stars/smallStar.json", "mode": "reference",
      "transform": { "angle": 30, "tx": 200, "ty": 0 } }
  ],
  "order": ["L2", "L1"] }
```

9.0 CHANGES FROM VERSION 1

Area	Version 1	Version 2	Solves
Geometry	Exists while painting; `draw()` and `infrastructure()` must agree	`build()` returns all geometry as data, once	P-1
Objects	Shared singletons with live settings; `Object.create` copies	Generators hold no state; each layer has its own settings in the document	P-2
Settings	`params` with a `points[]` mirror and derived values, all saved	Settings only; handles name point numbers; nothing derived is saved	P-3
Dragging	`update()` guesses which handle moved	A drag writes one named setting or point through a command	P-4
Handles	Undeclared; found by coordinate matching	Declared by name and role; `drag()` for unusual ones	P-5
Welds	Groups of copies kept in step by a propagation walk	Shared point numbers and one-way attachments; evaluation in order	P-6
Undo	None	Snapshot history; one gesture, one step	P-7
State	One object per tab mixing three kinds	Document, session and interaction kept apart	P-8
Tab switching	Decision order of flags	Close, then open from own state	P-9
Lists	Stored manifests, rebuilt and checked	Index worked out from folders plus a metadata file per item	P-10
Files and Node	Two boundaries, often bypassed	One Storage module, checked automatically	P-11
Scripts	Shared globals and state	Isolated frame, fresh canvas per run	P-12
Adding items	Restart or Rebuild; registry list edited by hand	Save the file; it appears	P-13
Controls	Two blocks; widget names; aliases; widgets write settings	One typed schema; widgets send commands	P-14
Output	Canvas pixels; no true-scale print	Units, SVG, calibrated PDF, nail list, spacing check	P-15
Menus, words	Eight menu files; "overlay" three ways	One commands table; panel and canvas layer	P-16
Metadata	Seven unread fields; decorative version	Only fields that are read; version drives migration	P-17
Parametrics	Separate code; spacing tools for ellipses only	A maker like any other; shapers work on every path	P-18
Adapting scripts	By hand, into a second format	Importer proposes levels 1 to 3; owner approves	P-19

What is retired. `drawState.js` globals; `window.drawRegistry` and its hand-kept list; manifests as authority, `maintain_manifests`, the start-up rebuild and the Home alert bar's reason for existing; `fileLayer` and direct `nodeDispatch` calls; the weld propagation walk and `cornerPartners()`; `needsUpdate`; the per-tab menu files; TinyMCE.

What is kept, by intent. The six tabs; Notes from header comments; permanent ids; stable numbers; layer transforms; infrastructure (now paths); the list builder's isolation; the playpen rule; the local Node service and its path safety; plain JavaScript.

10.0 CONSTRUCTION PLAN

10.1 Method

V2 is built in nine checkpoints on the version-2 branch. Each checkpoint is a working build that the owner can start on the PC and use, not a progress report. Work stops at each checkpoint for the owner's evaluation; the next begins only on the owner's word, as with V1 phases. V1 stays usable throughout. Each checkpoint ends with its tests passing and the owner's approval, at which point its output becomes the baseline (D-4).

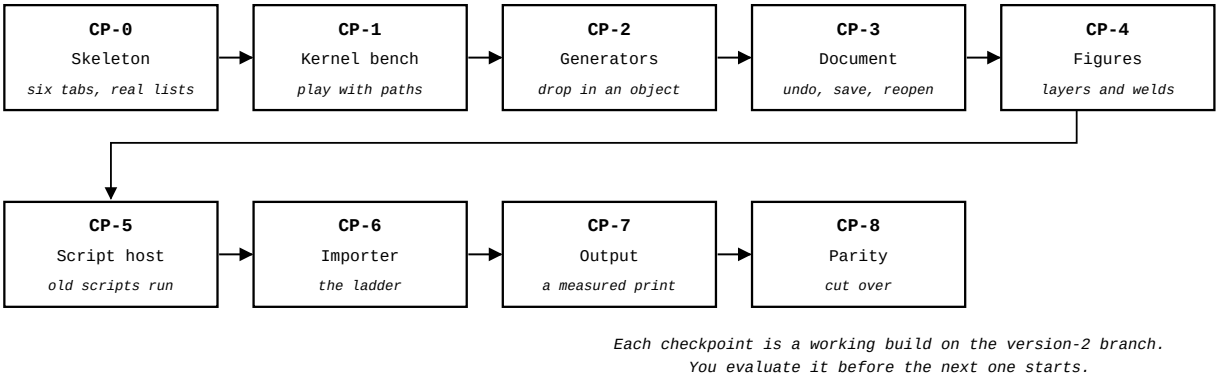


Figure 20. Checkpoints and their order

10.2 Before CP-0: feature lists

For each of the 14 Draw objects, a one-page feature list is drafted from V1: controls, each handle and what it does, infrastructure, and special behaviour (the Multi handle that separates copies; Linked Circles' three link styles; Chords in Ellipse's five modes). The owner corrects them. They record what each object is for, not what it outputs.

10.3 Checkpoints

CP-0 SKELETON	
Built	The `app2/` folder; one Node process serving the page and the requests; Bootstrap, Tab lifecycle, Panels, Library view; Storage (read only) and Catalog reading the existing folders; the call-rule check (FR-6.1.4).
You evaluate	Start V2 beside V1. Click through the six tabs. Each category list shows your real patterns, gallery scripts and images by name. Nothing runs yet.
Accepted when	Every category V1 shows is present; switching tabs leaves nothing behind; the call-rule check passes.
Not yet	Drawing, scripts, thumbnails, saving.

CP-1 KERNEL BENCH	
Built	The kernel (paths, makers, shapers, connectors, finders); the canvas renderer; a Utilities page, the *kernel bench*, with controls to choose a maker, chain shapers and pick a connector; Node tests for the kernel.
You evaluate	Make a circle of 36 points and connect by multiply 2 (a cardioid), then by all pairs. Make a rose with parametric, compare even t with even spacing. Add jitter, change the seed, change it back: the same picture returns. Trim, reverse, start at 4.
Accepted when	You can build each of Figure 8's four pictures on the bench, and the shapers behave as you expect.
Not yet	Generators, handles, documents.

CP-2 GENERATORS AND CONTROLS	
Built	The generator format and its load check; the settings schema and Controls; handles on the canvas through Interaction; the folder watcher. Generators: Line, Parabola, Circular Parabola Multi, Linked Circles. The Draw tab using them.
You evaluate	Open each object in Draw: sliders, number boxes, reset; drag handles; nudge with arrow keys. Save a copy of `parabola.js` under a new id with a change: it appears in Draw within two seconds. Edit it in VS Code and save: the open drawing redraws with your settings kept. Break a file on purpose: it shows a badge, the rest load.
Accepted when	The four objects match their feature lists; adding a generator needs nothing but saving the file.
Not yet	Undo, saving, figures.

CP-3 DOCUMENT, COMMANDS AND UNDO	
Built	Document, Commands and History, Session, Storage writes; the format-2 file; Save, Save As (one form), open; the unsaved indicator; the remaining ten generators.
You evaluate	In Draw, change settings and drag: Ctrl+Z undoes a whole slider movement or drag in one step; Ctrl+Y redoes. Save; undo back past the save and forward again: the indicator follows. Switch tabs and return: history is still there. Open the saved file in VS Code and read it.
Accepted when	All 14 objects match their feature lists; undo never leaves a wrong picture; saved files contain no derived values.
Not yet	Layers, welds, scripts.

CP-4 FIGURES	
Built	The Figures tab: layers, layer strip, identity colours, transforms, the point table, shared-point welds, attachments, discs, the Evaluator's ordering, break apart, nesting, roles; the commands table for Figures.
You evaluate	Build the *hard test figure*: two copies of one object; a hidden layer; a deleted layer; a moved and a turned layer; a free disc and an anchored disc; a corner tied within one layer; a weld across two layers; a line attached to a circle. Drag everything. Undo everything. Save, close, reopen. Try to make an attachment loop.
Accepted when	Every item of the hard test figure behaves, survives save and reopen, and undoes in one step per gesture; the loop is refused with a clear message.
Not yet	Scripts, output.

CP-5 SCRIPT HOST	
Built	The isolated frame; the legacy adapter; the recorder; error badges and time limit; reports and thumbnails on first sight; Patterns and Gallery running scripts.
You evaluate	Run a dozen of your scripts in any order, including the two that failed after about 100 runs in V1, `arcsAndInner`, `voronoi` and an animated one. Leave an animation: it stops. Read a report. Place a recorded script in a figure as a frozen layer.
Accepted when	Run order no longer matters; a broken script cannot affect another; every legacy script is classified as runs, adapted, preview only, or blocked, with a reason, in a migration ledger.
Not yet	The importer.

CP-6 IMPORTER	
Built	The importer with levels 1 to 3 and the review panel; New script and Import.
You evaluate	Take `OverlappingStars.js` through Level 1 and compare the proposal with the V1 hand adaptation. Adjust ranges, accept, and check the original is kept. Take one simple script through Level 3 and insert it into a figure.
Accepted when	Level 1 proposals need only small corrections on your test scripts; nothing is ever rewritten without your approval.
Not yet	Output.

CP-7 OUTPUT	
Built	Units and scale; output preview; SVG; true-scale PDF with scale bar and 100 mm calibration line; nail list; spacing check; tiling if wanted (O-7).
You evaluate	Set the scale for a figure. Print the PDF at actual size on your printer and measure the calibration line with a ruler. Check the nail list against the drawing. Set a nail diameter and see close pairs highlighted.
Accepted when	The printed 100 mm line measures 100 mm; a weld group prints as one nail; the template is usable on a board.
Not yet	Home, Utilities, Help parity.

CP-8 PARITY AND CUT-OVER	
Built	Home with status views; Utilities (tools and lab); Help in Markdown; thumbnails for all items; the parity ledger complete.
You evaluate	Use V2 for a week of real work. List anything V1 did that V2 does not.
Accepted when	The ledger shows every V1 item accounted for and you choose to retire V1.
Then	V2 is merged; `app2/` becomes the application; V1's code is removed in a separate, reviewable commit.

11.0 VERIFICATION

FR-11.1	Kernel, evaluator, commands, history, schema and storage shall have Node tests that run in seconds without a browser.
FR-11.2	Tests shall check meaning before pictures: path lengths, stitch end points, bounds, attachment positions, weld coincidence, undo restoring every partner.
FR-11.3	Each generator shall have, once the owner approves it, stored reference geometry and a reference picture. Later changes shall be compared with these; a deliberate change updates them.
FR-11.4	A browser test shall cover each screen's wiring and each checkpoint's evaluation steps that can be automated.
FR-11.5	Script tests shall run scripts in random order to prove run order no longer matters.
FR-11.6	Tests that save shall use a scratch copy, never the owner's library (V1's `SKETCHPAD_ROOT` rule kept).
FR-11.7	Physical output shall be accepted only by a measured print (CP-7).

12.0 OPEN DECISIONS AND ITEMS TO LOOK AT

12.1 Decisions needed from the owner

No.	Decision	Needed by
O-1	Units for new documents (mm or inches), and how a V1 drawing's scale is set: one measurement you supply, as R-7 recommends, rather than a fixed guess.	CP-3
O-2	Whether coincident points that are not welded print as one nail or two.	CP-7
O-3	Nail diameter and minimum clearance for the spacing check.	CP-7
O-4	Whether to use Acorn, a small outside JavaScript parser kept in `vendor/`, for the importer. It is code you did not write; writing a parser instead would be far worse.	CP-6
O-5	Whether lists need hand-set order or hidden entries (fields in the metadata file).	CP-0
O-6	Whether choosing a layer should still bring it to the front.	CP-4
O-7	Board sizes, and therefore whether tiling across sheets is needed.	CP-7
O-8	What Parametrics may drag (it has no handles in V1), and whether `boxes.js` comes across at all.	CP-3
O-9	The Sweep tool (Phase 13) has no definition on record. R-7 suggests a parameter study: a grid of drawings over a range of one setting.	after CP-8
O-10	Crossings as nail points (October item I2): finders can name them; whether they print is your call.	CP-7

12.2 From the other evaluations: worth your attention

Four evaluations were considered (R-5, R-6, R-7, and the staged rethink R-4). Most of their substance is in this specification. These items are not, or are only in part, and are worth a look.

Source	Item	Why it matters	Status here
R-7	**The physical acceptance test**: print at actual size and measure a calibration line.	The only real proof a template is right; a screen preview cannot prove it.	**Adopted** (CP-7). Look at R-7 p. 12.
R-7	**The hard test figure** and the migration ledger.	A single pretty shape exercises nothing; the ledger accounts for all 185 scripts.	**Adopted** (CP-4, CP-5).
R-7	**Snap versus attach**: snapping aligns once; attaching persists.	Both are useful and easy to confuse; the difference should be visible.	Partly. Attach is specified; a one-time snap is **worth deciding**.
R-7	Deterministic results and residual-error reporting.	Identical edits must give identical results. With ordered evaluation this is natural; reporting where an attachment could not be placed matters.	Adopted in FR-6.5.9.
R-7	Browser storage, crash recovery, revision conflicts, touch authoring, portable bundles.	Valuable for authoring on many devices.	**Set aside** by D-2 (iPad for review only). Revisit only if that changes.
R-6	**Output engines** beyond PDF: plotter (HPGL), laser or vinyl cutter, CNC drilling, DXF for CAD.	A plotter could draw stitch lines; a CNC router could drill the holes. Each engine is one file once geometry is data.	**Worth a look**. Not specified; add any you would use (R-6 section 10).
R-6	**Type checking without a build**: `// @ts-check` and JSDoc comments, so VS Code underlines wrong property names as you type.	Catches the misspelled-field class of error; files stay plain JavaScript.	**Worth a look**. Optional under A-2.
R-6	One Node process serving both the page and the requests.	One window, one port, one command.	Adopted in CP-0.
R-6	3D string art (cylinders, spheres, tubes) as a later phase.	Linked Circles already suggests tubes and rings; purpose 2.	Not specified. A separate project if wanted.
R-4	Staged evolution of V1 instead of a rebuild.	Lower risk, nothing stops working.	Superseded by D-1. R-6's step table remains a fallback route.
R-8	October items deferred then: blend modes (L1), background as a layer (L2, N14), hover information (B20), a context bar (O1), node metadata (L9).	Some fit naturally now: background is the image role; node metadata is a field on a point-table entry.	**Worth a look** before CP-4.

13.0 GLOSSARY

Attachment	A point-table entry that lies on another layer's path; one-way.
Catalog	The index of library items, worked out from the folders.
Checkpoint	A working build the owner evaluates before work continues.
Command	A named change to a document; the only way a document changes.
Connector	A kernel rule joining path points into stitches.
Disc	A numbered point placed on a layer: free, or anchored to a path (an attachment).
Document	Plain JSON describing a drawing or figure; saved; undoable.
Finder	A kernel rule naming points on a path by feature.
Generator	One file describing one drawable object or script: meta, settings schema, build or run.
Handle	A draggable setting of type point; it names a point in the point table.
Kernel	The geometry code: values in, values out, no state, no painting.
Maker	A kernel rule producing a path: line, circle, parametric curve and so on.
Panel	A side area of the page. Replaces one meaning of V1's "overlay".
Path	An ordered array of points, open or closed.
Point table	The document's list of numbered points; welds are shared entries.
Renderer	The only code that paints: canvas, SVG, PDF, nail list.
Session	View state: selection, mode, zoom. Not saved, not undone.
Shaper	A kernel rule turning a path into another path.
Weld	Two handles naming one point (shared), or a handle attached to a path.

APPENDIX A. TRACEABILITY

Problem	Requirements	Checkpoint
P-1 geometry while painting	FR-6.2.9, 6.3.4, 6.11.1, 6.11.8	CP-1, CP-2
P-2 shared singletons	FR-6.3.1, 6.5.7	CP-2
P-3 mixed settings	FR-6.4.1, 6.5.7	CP-3
P-4 guessing drags	FR-6.3.5, 6.3.6, 6.4.4	CP-2
P-5 undeclared handles	FR-6.3.5 to 6.3.7	CP-2
P-6 weld propagation	FR-6.5.1 to 6.5.5	CP-4
P-7 no undo	FR-6.6.1 to 6.6.8	CP-3
P-8 mixed state	FR-6.7.1 to 6.7.4	CP-3, CP-4
P-9 tab flags	FR-6.7.5, 6.7.6	CP-0
P-10 manifests	FR-6.8.1 to 6.8.4	CP-0, CP-2
P-11 bypassed boundaries	FR-6.1.4, 6.8.5	CP-0
P-12 shared script state	FR-6.10.1 to 6.10.3	CP-5
P-13 adding items	FR-6.9.1 to 6.9.8	CP-2
P-14 controls	FR-6.4.1 to 6.4.8	CP-2
P-15 units and output	FR-6.5.8, 6.11.4 to 6.11.7	CP-7
P-16 menus and words	FR-6.12.2, 6.12.3	CP-4, CP-8
P-17 metadata	FR-6.3.2, 6.3.3, 6.5.6	CP-2, CP-3
P-18 parametric divide	FR-6.2.2, 6.2.3, 6.2.8	CP-1
P-19 manual adaptation	FR-6.10.5 to 6.10.10	CP-6

END OF DOCUMENT